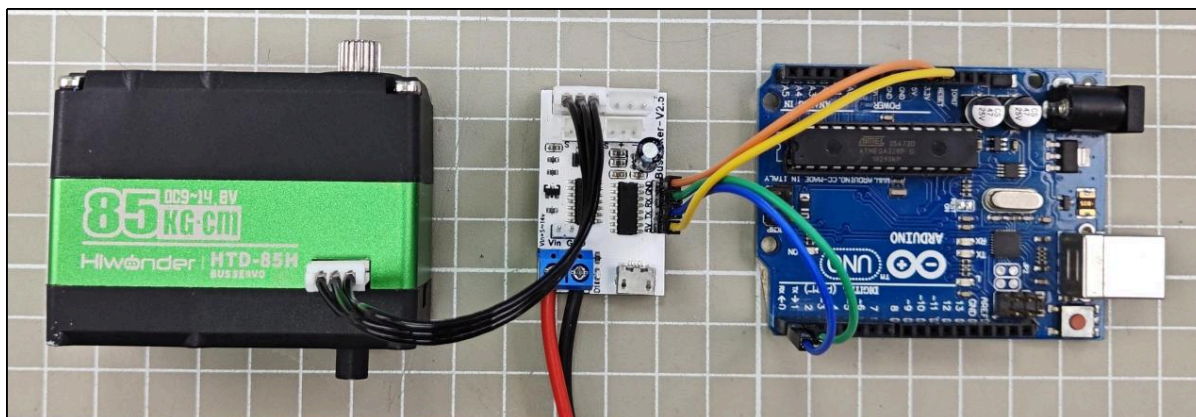


2. 总线舵机二次开发教程-Arduino版本

2.1 准备工作

2.1.1 接线说明

本节示例使用的是Arduino UNO和总线舵机调试板进行开发，通过11.1V 6000mAh锂电池来供电。将总线舵机连接至总线舵机调试板任意一个总线接口，再将总线舵机调试板的串口连接至Arduino的串口(5V--5V、RX--TX、TX--RX、GND--GND)。



Note

注意：

1. 如使用我司锂电池，连接锂电池对接线请以红接+，黑接-接到DC接口。
2. 如对接线未连接锂电池，请勿与电池对接线直接对接，避免正负极发生短路而造成短路。

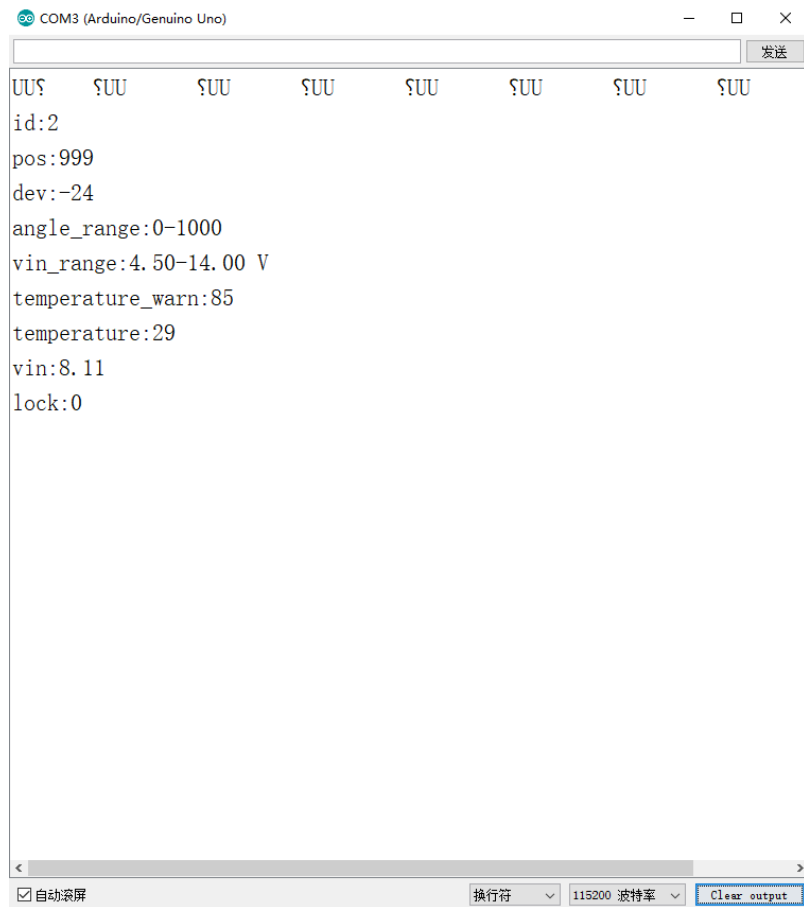
2.1.2 环境配置

在电脑端安装Arduino IDE，软件包位于“2 软件工具->Arduino环境搭建”下。关于Arduino IDE的详细使用，可在对应目录下进行学习。

2.2 案例开发

2.2.1 案例1 总线舵机信息读取

本案例通过终端窗口显示出总线舵机ID、位置、温度等相关信息。



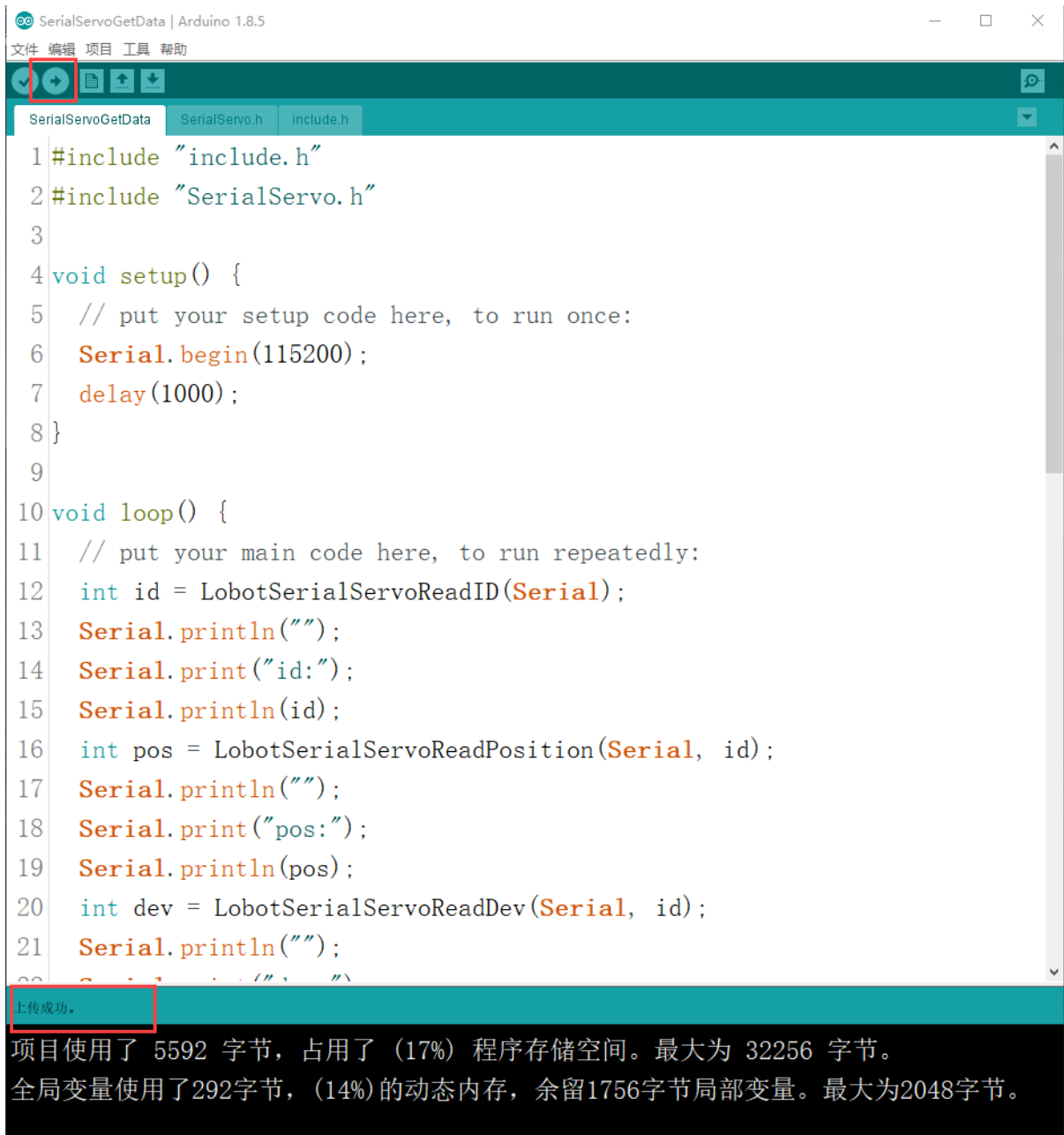
- 运行程序

在“01 Arduino\案例1 总线舵机信息读取\SerialServoGetData”路径下双击打开“SerialServoGetData.ino”程序。

将Arduino UNO连接至电脑，点击“”下载程序。

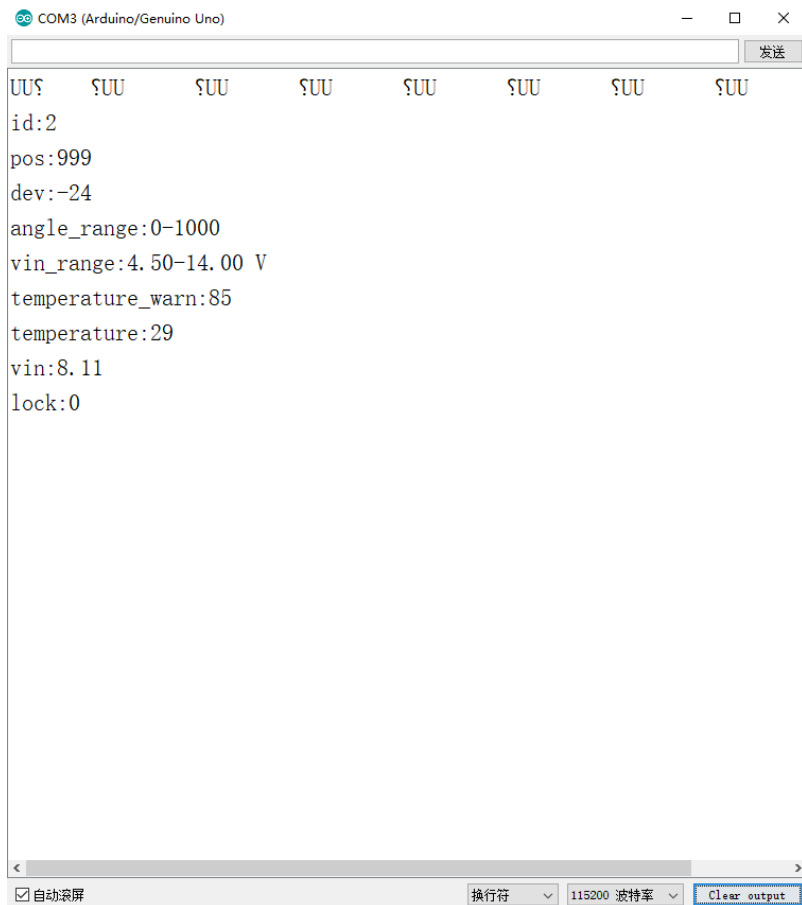
Note

注意：如果出现上传失败的提示，可尝试将舵机调试板断开Arduino后上传，上传完毕后再重新接入Arduino。



- 实现效果

程序运行后，终端打印画面会滚屏打印舵机各项状态信息。



1. `id`:舵机ID。在这里示例为2。
2. `pos`:舵机当前所在的位置。在这里示例为999。
3. `dev`:舵机偏差。在这里示例为-24。
4. `angle range`:舵机限位范围。在这里示例为0-1000。
5. `voltage range`:舵机电压范围。在这里示例为4.5~14V。
6. `temperature warn`:舵机过温报警阈值。在这里示例为85℃。
7. `temperature`:舵机当前温度。在这里示例为29℃。
8. `vin`:舵机当前电压值。在这里示例为8.11V。
9. `lock`:舵机是否为上电状态。在这里示例为0，代表为掉电状态。当为1，即上电状态。

Arduino IDE串口监视器打印的信息中如果存在乱码的原因：

由于舵机调试板与USB共用同一个串口，所以在Arduino给舵机调试板发送数据时，我们的PC端也能接收到数据，所以打印出的乱码是Arduino发送给舵机调试板的数据包，我们可以忽略这些信息。

- 案例程序简要分析

(1)导入必要功能包

```
#include "include.h"
#include "SerialServo.h"
```

两个功能包在程序文件同路径下，主要封装用于总线舵机通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

(2)串口初始化操作

```
void setup() {  
    Serial.begin(115200);  
    delay(1000);  
}
```

总线舵机通信波特率固定为115200，在与舵机进行通信之前需要将Arduino串口波特率设置为115200。

(3)获取舵机状态信息并打印

```
int id = LobotSerialServoReadID(Serial); //读取当前舵机ID  
int pos = LobotSerialServoReadPosition(Serial, id); //读取舵机实时位置  
int dev = LobotSerialServoReadDev(Serial, id); //读取舵机偏差，默认0  
LobotSerialServoReadAngleRange(Serial, id); //读取舵机角度限位，默认(0-1000)  
LobotSerialServoReadVinLimit(Serial, id); //读取电压限制范围，默认(4.50-14.00v)  
int temperature_warn = LobotSerialServoReadTempLimit(Serial, id); //读取温度报警阈值，默认(85°)  
int temperature = LobotSerialServoReadTemp(Serial, id); //读取舵机实时温度  
int vin = LobotSerialServoReadVin(Serial, id); //读取舵机实时电压  
int lock = LobotSerialServoReadLoadOrUnload(Serial, id); //读取舵机状态
```

通过调用上述函数，获取舵机的各种状态信息。这些状态信息包括舵机ID、位置、偏差、角度范围、电压范围、过温报警阈值、当前温度、电压以及舵机上电状态。

```
Serial.print("id:");  
Serial.println(id);  
Serial.print("pos:");  
Serial.println(pos);  
Serial.print("dev:");  
Serial.println(dev);  
Serial.print("angle_range:");  
Serial.print(retL);  
Serial.print("-");  
Serial.println(reth);  
Serial.print("vin_range:");  
Serial.print((float)vinL/1000);  
Serial.print("-");  
Serial.print((float)vinH/1000);  
Serial.println(" v");  
Serial.print("temperature_warn:");  
Serial.println(temperature_warn);  
Serial.print("temperature:");  
Serial.println(temperature);  
Serial.print("vin:");  
Serial.println((float)vin/1000);  
Serial.print("lock:");  
Serial.println(lock);
```

在获取后，可将上面的这些参数打印出来。每条信息打印最后都会使用println进行换行，以对齐输出结果。

2.2.2 案例2 总线舵机ID设置

本案例修改舵机ID，并通过终端窗口显示出总线舵机新的ID。



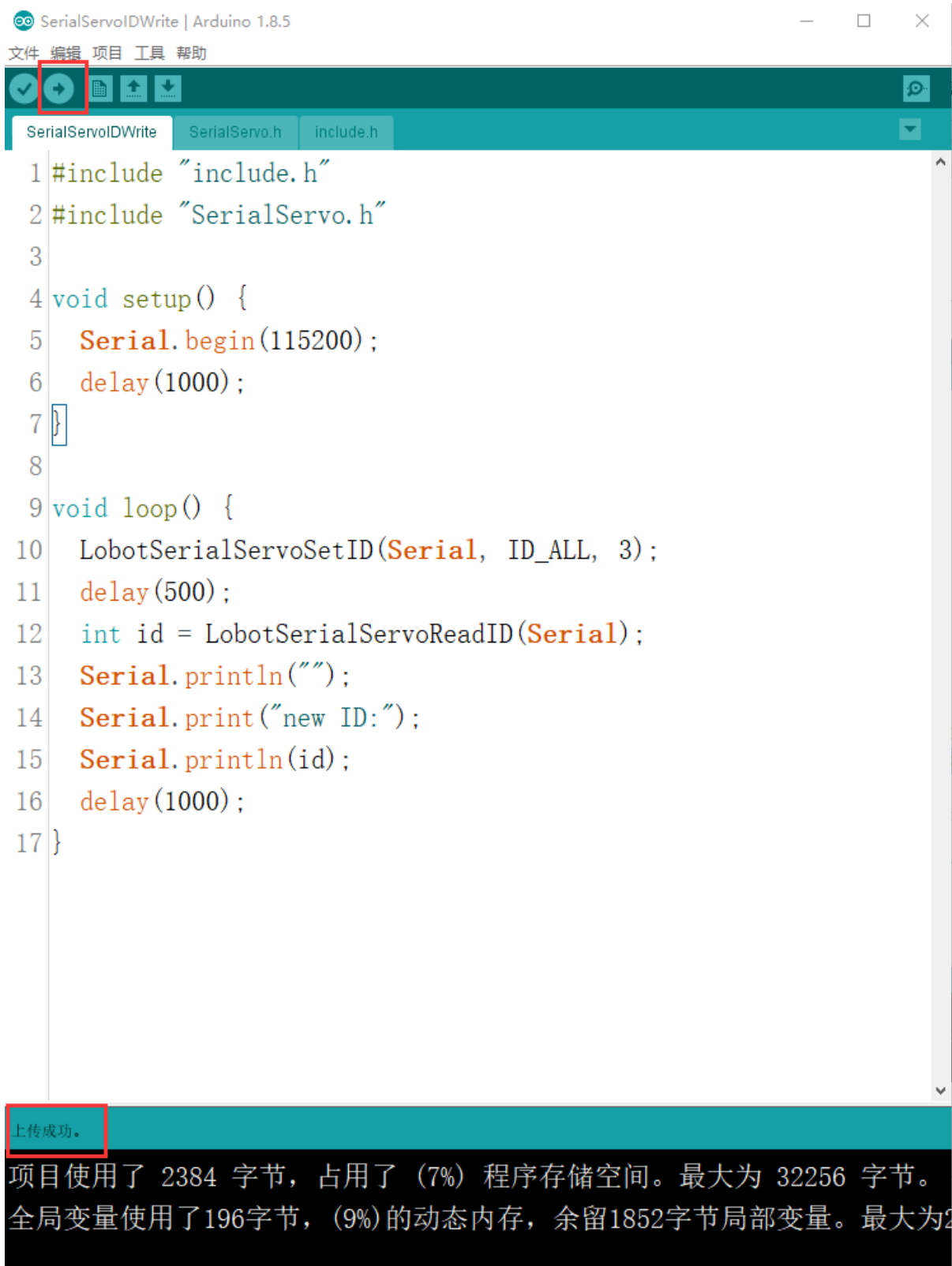
- 运行程序

在“01 Arduino\案例2 总线舵机ID设置\SerialServoIDWrite”路径下双击打开“SerialServoIDWrite.ino”程序。

将Arduino UNO连接至电脑，点击“”下载程序。

Note

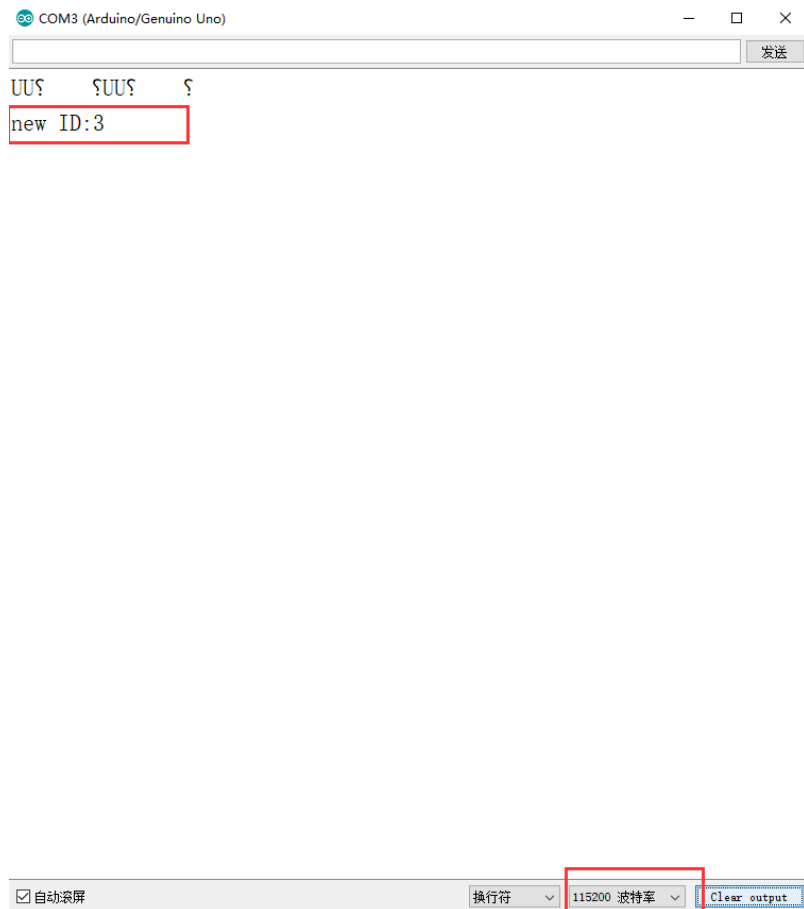
注意：如果出现上传失败的提示，可尝试将舵机调试板断开Arduino后上传，上传完毕后再重新接入Arduino。



上传成功后，点击右上角“”的串口监视器，将波特率设置为115200，即可看到打印的信息。

- **实现效果**

程序运行后，会通过终端窗口显示出总线舵机新的ID。



各项信息的具体含义如下：

`new ID`:修改后的舵机ID。在这里示例为3。

串口监视器打印的信息中如果存在乱码的原因：

由于舵机调试板与USB共用同一个串口，所以在Arduino给舵机调试板发送数据时，我们的PC端也能接收到数据，所以打印出的乱码是Arduino发送给舵机调试板的数据包，我们可以忽略这些信息。

- 案例程序简要分析

(1)导入必要功能包

```
#include "include.h"
#include "SerialServo.h"
```

两个功能包在程序文件同路径下，主要封装用于总线舵机通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

(2)串口初始化操作

```
void setup() {
    Serial.begin(115200);
    delay(1000);
}
```

总线舵机通信波特率固定为115200，在与舵机进行通信之前需要将Arduino串口波特率设置为115200。

(3)修改舵机ID信息

```
LobotSerialServoSetID(Serial, ID_ALL, 3);  
delay(500);
```

通过调用 `LobotSerialServoSetID()` 函数，将连接在总线舵机调试板上的舵机的ID值修改为3。此处 `ID_ALL` 变量的值为254，在总线舵机通信协议中表示为广播ID，可用于对未知ID的舵机读取ID值。

```
int id = LobotSerialServoReadID(Serial);  
Serial.println("");  
Serial.print("new ID:");  
Serial.println(id);  
delay(1000);
```

通过读取舵机ID，将刚刚修改的舵机ID打印出来。

2.2.3 案例3 控制总线舵机转动

本案例使用Arduino UNO控制舵机从位置0-1000以1秒间隔来回转动。

- 运行程序

在“01 Arduino\案例3 控制总线舵机转动\SerialServoRP”路径下双击打开“SerialServoRP.ino”程序。

将Arduino UNO连接至电脑，点击“”下载程序。

Note

注意：如果出现上传失败的提示，可尝试将舵机调试板断开Arduino后上传，上传完毕后再重新接入Arduino。



上传成功后，将舵机调试板接到Arduino上，舵机开始转动。

- 实现效果

程序运行后，舵机会从位置0-1000以1秒为间隔来回转动。

- 案例程序简要分析

(1)导入必要功能包

```
#include "include.h"
#include "SerialServo.h"
```

两个功能包在程序文件同路径下，主要封装用于总线舵机通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

(2)串口初始化操作

```
void setup() {  
    Serial.begin(115200);  
    delay(1000);  
}
```

总线舵机通信波特率固定为115200，在与舵机进行通信之前需要将Arduino串口波特率设置为115200。

(3)控制舵机转动

```
void loop() {  
    LobotSerialServoMove(Serial, ID_ALL, 0, 1000);  
    delay(1000);  
    LobotSerialServoMove(Serial, ID_ALL, 1000, 1000);  
    delay(1000);  
}
```

通过调用 `LobotSerialServoMove()` 函数，控制舵机转动。上述过程主要实现了，舵机以1秒的时间转动到位置0，延时1秒后，再以1秒的时间转动到位置1000。

舵机转动范围为：0-1000，对应角度为：0°-240°。

其中 `ID_ALL` 变量的值为254，在总线舵机通信协议中表示为广播ID，可用于对未知ID的舵机进行位置控制。

2.2.4 案例4 调节总线舵机速度

本案例通过Arduino控制舵机以不同的速度进行转动。

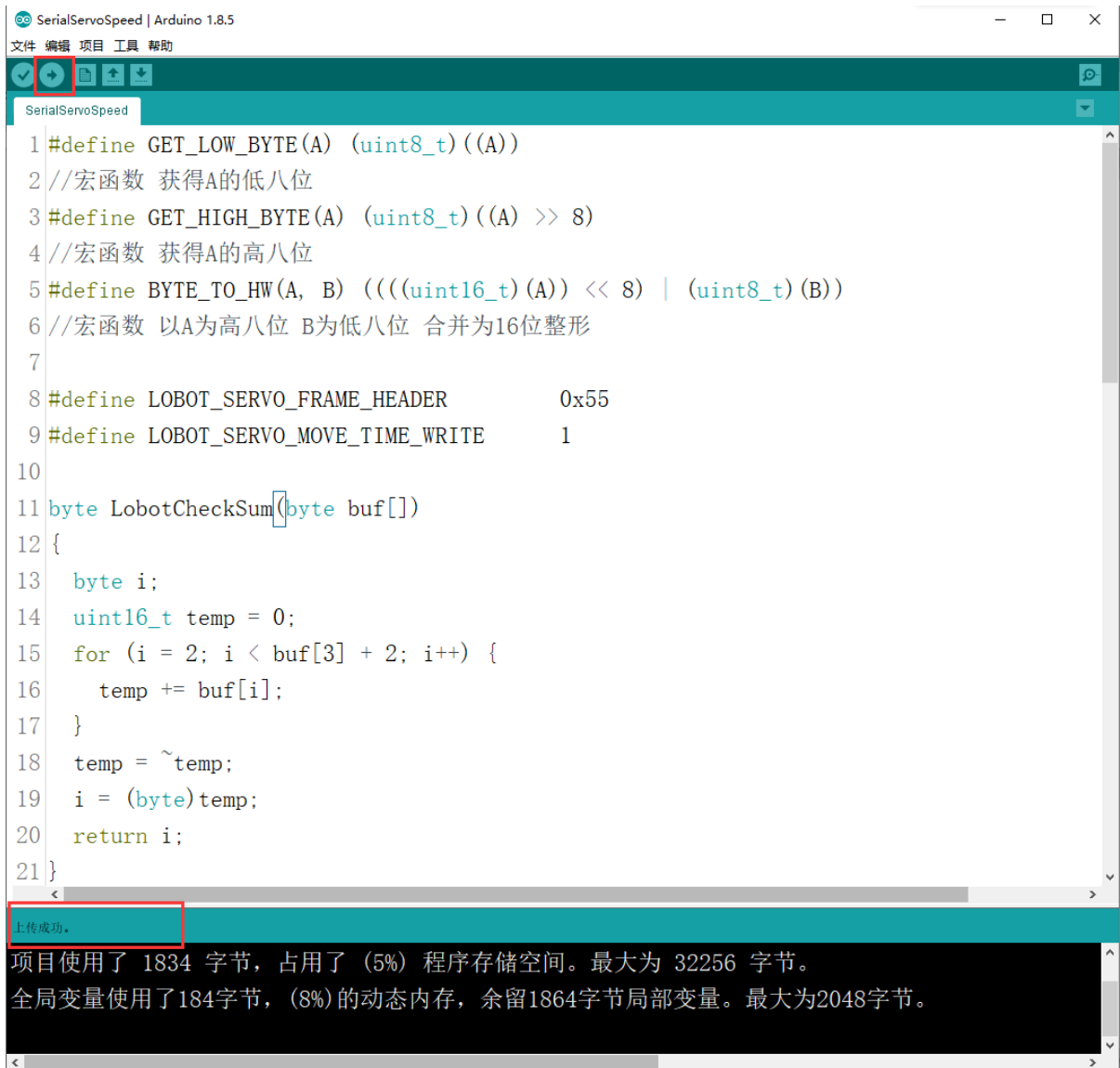
- 运行程序

在“01 Arduino\案例4 控制总线舵机速度\SerialServoSpeed”路径下双击打开“SerialServoSpeed.ino”程序。

将Arduino UNO连接至电脑，点击“”下载程序。

Note

注意：如果出现上传失败的提示，可尝试将舵机调试板断开Arduino后上传，上传完毕后再重新接入Arduino。



上传成功后，将舵机调试板接到Arduino上，舵机开始以不同的速度转动。

• 实现效果

程序运行后舵机现象会循环如下：

1. 舵机从位置0开始；
2. 以1秒的时间转动到位置500；
3. 以0.5秒的时间转动到位置1000；
4. 以1秒的时间转动到位置500；
5. 以0.5秒的时间转动到位置0。

• 案例程序简要分析

(1)导入必要功能包

```
#include "include.h"
#include "SerialServo.h"
```

两个功能包在程序文件同路径下，主要封装用于总线舵机通信的各功能模块，我们可以使用其中定义的变量和函数来控制舵机。

(2)串口初始化操作

```
void setup() {  
    Serial.begin(115200);  
    LobotSerialServoMove(Serial, ID_ALL, 0, 500);    //初始设置舵机位置为0  
    delay(1000);  
}
```

总线舵机通信波特率固定为115200，在与舵机进行通信之前需要将Arduino串口波特率设置为115200。初始设置舵机位置为：0。

(3)以不同的速度控制舵机转动

```
void loop() {  
    LobotSerialServoMove(Serial, ID_ALL, 500, 1000);  
    delay(1000);  
    LobotSerialServoMove(Serial, ID_ALL, 1000, 500);  
    delay(1000);  
    LobotSerialServoMove(Serial, ID_ALL, 500, 1000);  
    delay(1000);  
    LobotSerialServoMove(Serial, ID_ALL, 0, 500);  
    delay(1000);  
}
```

通过调用 LobotSerialServoMove() 函数，控制舵机转动。上述过程主要实现了，舵机以1秒的时间转动到位置500，延时1秒后，再以0.5秒的时间转动到位置1000，延时1秒后，再以1秒的时间转动到位置500，延时1秒后，再以0.5秒的时间转动到位置0。

舵机转动范围为：0-1000，对应角度为：0°-240°。

最后一个参数为转动到指定位置所需时间，参数值越小速度越快。

其中 ID_ALL 变量的值为254，在总线舵机通信协议中表示为广播ID，可用于对未知ID的舵机进行位置控制。

2.2.5 案例5 示教记录操作

本案例使用Arduino UNO与按键控制舵机，通过存储位置，让舵机转动到指定角度。

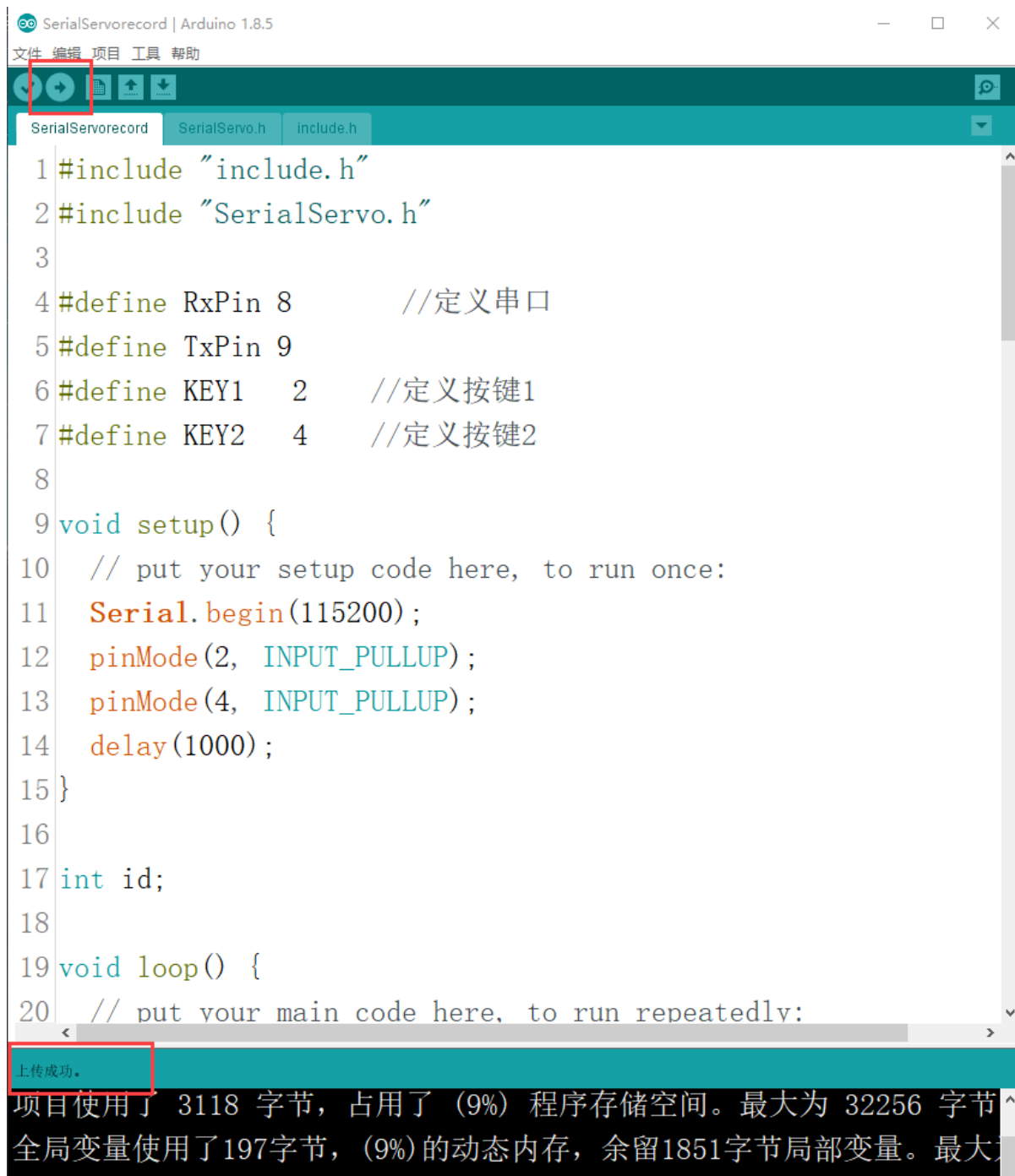
- 运行程序

在“01 Arduino\案例5 示教记录操作\SerialServorecord”路径下双击打开“SerialServorecord.ino”程序。

将Arduino UNO连接至电脑，点击“”下载程序。

Note

注意：如果出现上传失败的提示，可尝试将舵机调试板断开Arduino后上传，上传完毕后再重新接入Arduino。



• 实现效果

程序运行后, 按下按键1, 舵机掉电, 此时掰动舵盘至任意角度, 按下按键2记录当前位置(最多可记录4个位置, 如果超过4个将会从第一个位置开始覆盖)。当记录好位置后, 再次按下按键1, 此时舵机上电, 在上电状态按下按键2时, 舵机会按照记录好的角度从第一个开始依次进行转动。

• 案例程序简要分析

(1)导入必要功能包

```
#include "include.h"
#include "SerialServo.h"
```

两个功能包在程序文件同路径下, 主要封装用于总线舵机通信的各功能模块, 我们可以使用其中定义的变量和函数来控制舵机。

(2)串口初始化操作

```
#define KEY1 2 //定义按键1
#define KEY2 4 //定义按键2

void setup() {
    Serial.begin(115200);
    pinMode(2, INPUT_PULLUP);
    pinMode(4, INPUT_PULLUP);
    delay(1000);
}
```

定义两个按键的接口并设置为输入模式，总线舵机通信波特率固定为115200，在与舵机进行通信之前需要将Arduino串口波特率设置为115200。

(3)示教记录功能实现

```
static bool run = false;
static char step = 0;
static char mode = 0;
static int pos[4] = {100,200,300,400};
uint16_t temp;
id = LobotSerialServoReadID(Serial);
```

初始化程序所需的变量。

1. `pos[4]` 用来存储4个位置，方便在控制舵机转动的时候设置舵机位置；
2. `id` 为当前舵机ID；
3. `run` 为控制舵机转动标志位，当`run`为1时舵机开始转动；
4. `step` 用来表示舵机运行的第几个位置；
5. `mode` 表示程序当前模式。`mode`为0表示为控制模式，`mode`为1表示为读取模式。

```
if (mode == 0)
{
    if (run)
    {
        LobotSerialServoMove(Serial, id, pos[step++], 500);
        if (step == 4)
        {
            step = 0;
            run = false;
        }
        delay(1000);
    }
    if (!digitalRead(KEY2))
    {
        delay(10);
        if (!digitalRead(KEY2))
        {
            run = true;
        }
    }
}
```

```

        step = 0;
        delay(500);
    }
}
if (!digitalRead(KEY1))
{
    delay(10);
    if (!digitalRead(KEY1))
    {
        LobotSerialServoUnload(Serial, id);
        mode = 1;
        step = 0;
        delay(500);
    }
}
}
}

```

当程序为控制模式时，判断run标志变量，如果为1则控制舵机依次转动4个位置，完成后将run标志复位。

如果检测到按键2按下时，run标志变量设置为true，允许舵机转动。

如果检测到按键1按下时，mode=1，更改当前模式为读取模式，并使舵机卸载（掉电）。

```

if (mode == 1)
{
    if (!digitalRead(KEY2))
    {
        delay(10);
        if (!digitalRead(KEY2))
        {
            pos[step++] = LobotSerialServoReadPosition(Serial, id);
            if (step == 4)
                step = 0;
            delay(500);
        }
    }
    if (!digitalRead(KEY1))
    {
        delay(10);
        if (!digitalRead(KEY1))
        {
            temp = LobotSerialServoReadPosition(Serial, id);
            LobotSerialServoMove(Serial, id, temp, 200);
            mode = 0;
            delay(500);
        }
    }
}
}
}

```

当程序为读取模式时，可对存储的4个位置信息进行修改。当按键2按下时，记录当前舵机位置信息，存储在 pos[4] 数组中。当记录的位置信息超过4个时，会从第一个位置开始覆盖。

如果按键1按下后，将舵机上电，并控制舵机维持在当前位置。

